

图： 非线性、多对多的一种数据结构。

• 图的定义：  $G=(V,E)$ ，其中  $V=\{x|x \text{ 为图中的顶点}\}$ ； $E=\{(x,y)|x,y \in V\}$

有向图：  $\langle x,y \rangle$  顶点对是有序的； 否则为无向图

完全图： 若图有  $n$  个顶点，则无向完全图有  $\frac{n(n-1)}{2}$  条边；有向完全图有  $n(n-1)$  条边。

邻接顶点： 若  $(u,v) \in E$ ，则  $u,v$  互为邻接顶点

子图： 设  $G=(V,E)$ ， $G'=(V',E')$ ，若  $V' \subseteq V$  且  $E' \subseteq E$ ，则  $G'$  是  $G$  的子图

顶点的度： 与某个顶点关联的边的条数，记作  $deg(v)$  （有向图中分为入度和出度）

路径： 从一个顶点到另一个顶点经过的边的集合

路径长度： 若图带权，则为此路径上各边权值和； 否则为路径上边的条数

连通图、连通分支、生成树： 见“离散数学”中各定义

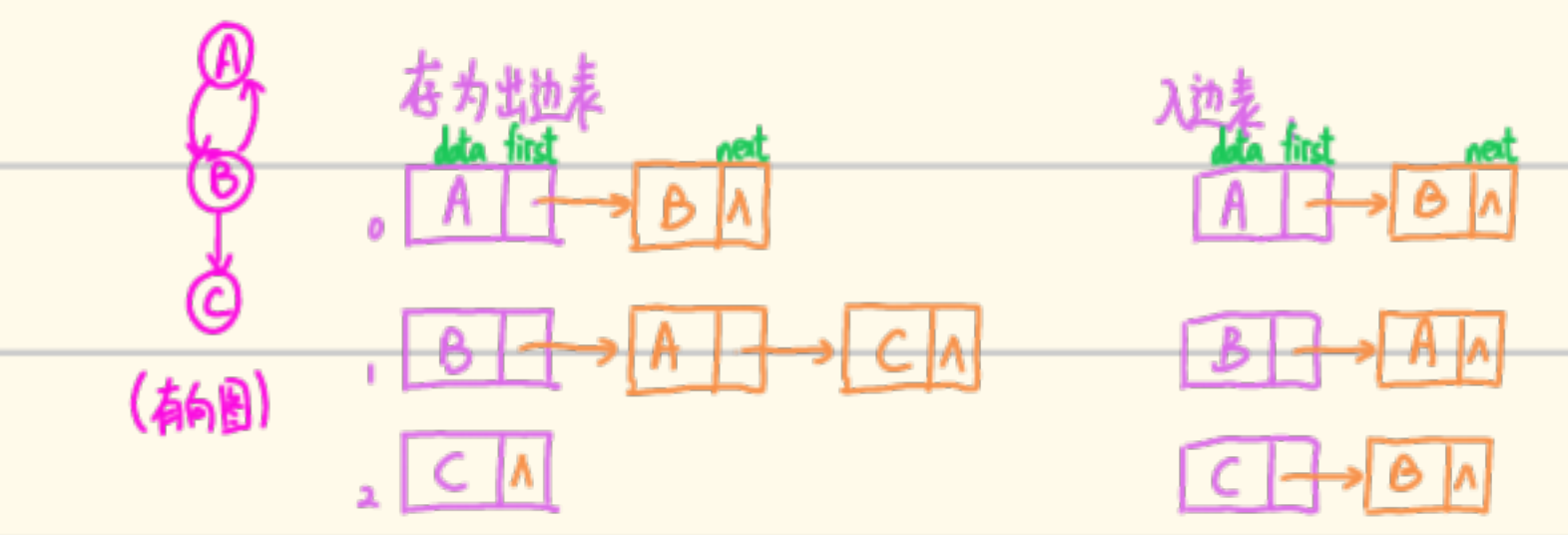
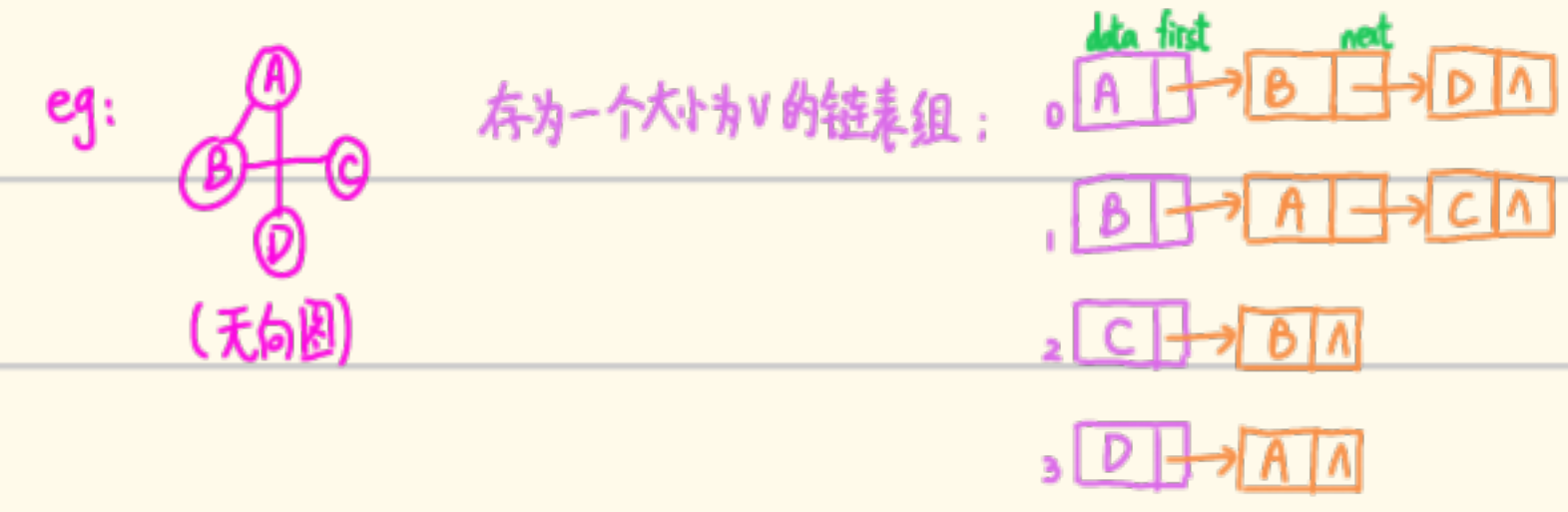
• 图的存储

• 邻接矩阵： 若顶点  $i$  到  $j$  有边，则矩阵  $(n \times n)$  中  $a_{ij}=1$  （无向图则  $a_{ij}, a_{ji}$  均为1）

\* 性质： 无向图的邻接矩阵是对称的；

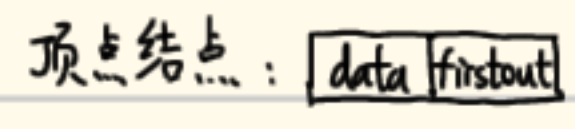
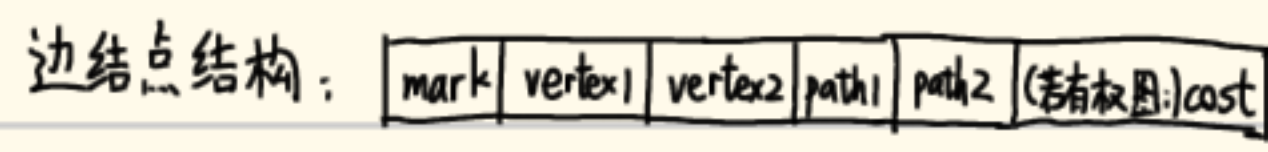
有向图中，第  $i$  行  $1$  的个数即顶点  $i$  的出度；第  $j$  列  $1$  的个数即顶点  $j$  的入度  
(对角线上即  $i=j$ ，全设为0)  
(带权图的邻接矩阵：  $a_{ij}$  等于顶点  $i \rightarrow j$  的边的权值，若  $i \rightarrow j$  无边则  $a_{ij}$  设为无穷大。)

• 邻接表： 省略了邻接矩阵中的“0”，将空间从  $O(v^2)$  降到  $O(v+e)$



出度、入度直接数对应顶点的那个链表有几个元素即得。

• 邻接多重表： 重在存边，避免重复处理且方便边操作（不考）



• 图的遍历： 从连通图的某一顶点出发，不重复地依次访问图中所有顶点。

(一). 深度优先搜索：

选定某一顶点  $v$  起始，访问其任一邻接顶点  $w_1$ ；再从  $w_1$  出发，访问任一与  $w_1$  邻接且未被访问过的邻接顶点  $w_2$ ；----- 直至接着找  $w_i$  没有未被访问过的邻接顶点；然后回溯，再同理找其他未被访问过的邻接顶点；----- 直至图中所有顶点都被访问过。

代码： (预先初始化 `bool visited[v]`)

```
void DFS(Graph& G, int v, bool visited[]) { //从v顶点开始DFS
    cout << G.getvalue(v) << " "; visited[v]=true; //访问编号为v的顶点
    int w = getFirstNeighbour(v); //找v的第一个邻居
    while (w != -1) { //只要有邻接顶点w
        if (!visited[w]) DFS(G,w, visited); //递归访问未遍历过的w
        w = G.getNextNeighbour(v,w); //v的其他邻接顶点
    }
}
```

(二). 广度优先搜索：

用辅助队列，先将起始顶点  $v$  进队，然后只要队非空(循环)，做如下操作：

{ 访问队头；标记为 `visited`；出队  
将队头元素的邻接顶点中未 `visited` 过的依次入队。

• 时间复杂度： 对 DFS 和 BFS，若用邻接表则均为  $O(n+e)$ ，  
若用邻接矩阵则均为  $O(n^2)$

• 最小生成树： 某带权图有  $n$  个顶点，选用  $n-1$  条边(无回路)构成生成树，使总权重最小

• kruskal 算法： (使用最小堆、并查集来实现)

先构造一个只有  $n$  个顶点，没有边的图，然后将所有边放在一个最小堆中。每次取堆顶，

从堆中将其删除。然后检查此边的两个顶点是否在同一条分支上(用并查集的 `find()` 操作)，

若不在则将此边加入树  $T$  中，并且 `Union` 两个点；否则直接不操作(相当于舍弃此边)。

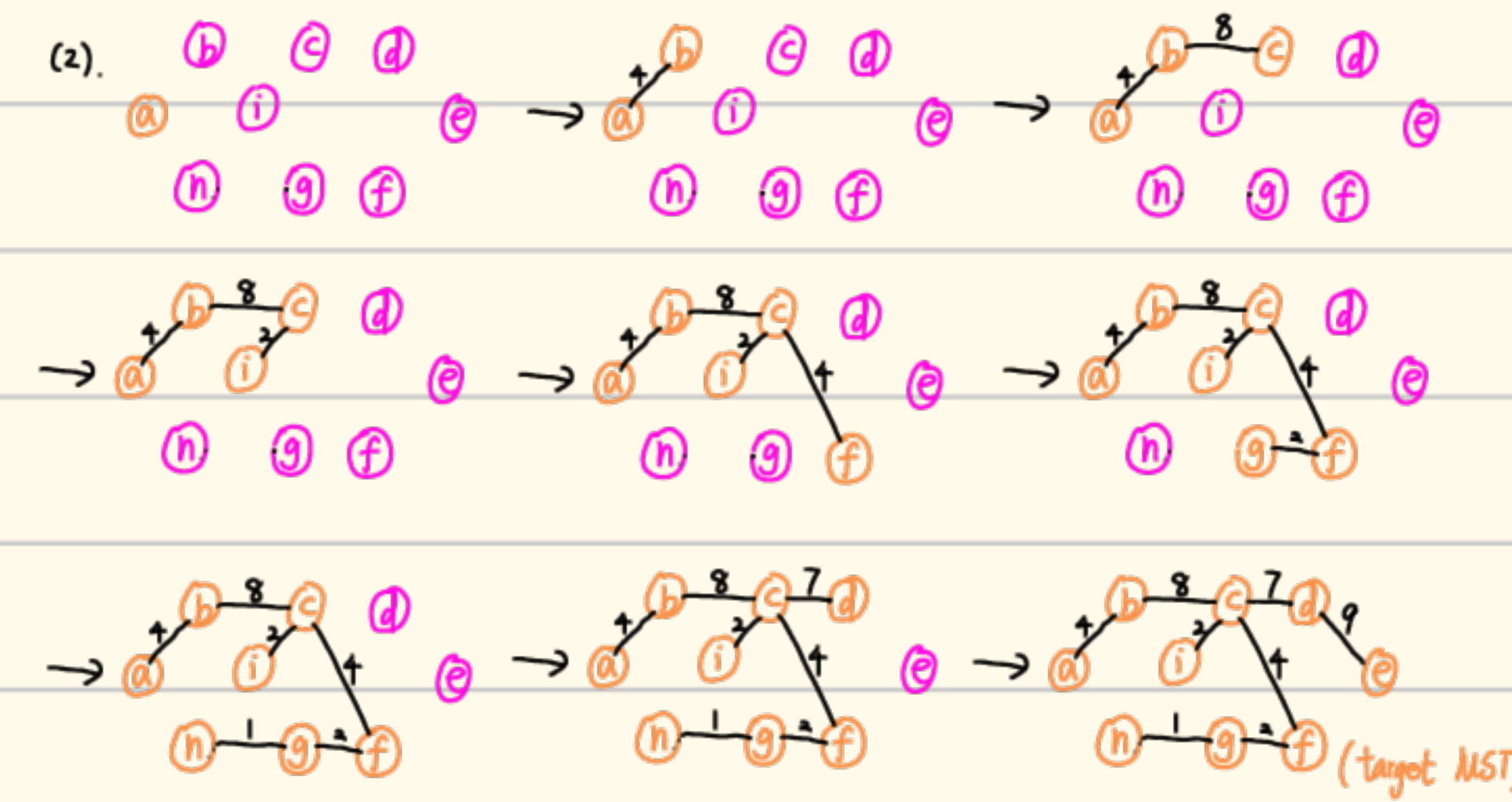
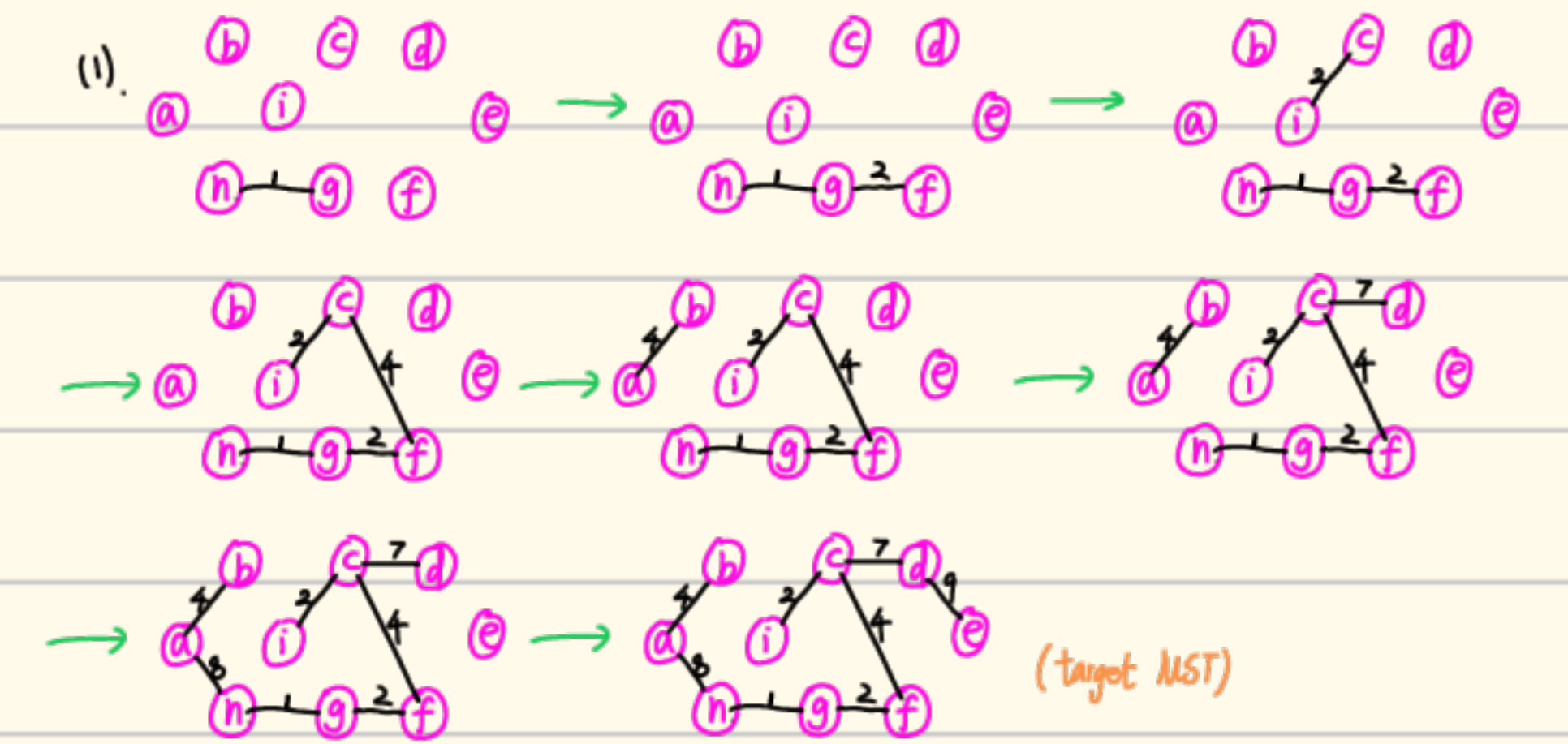
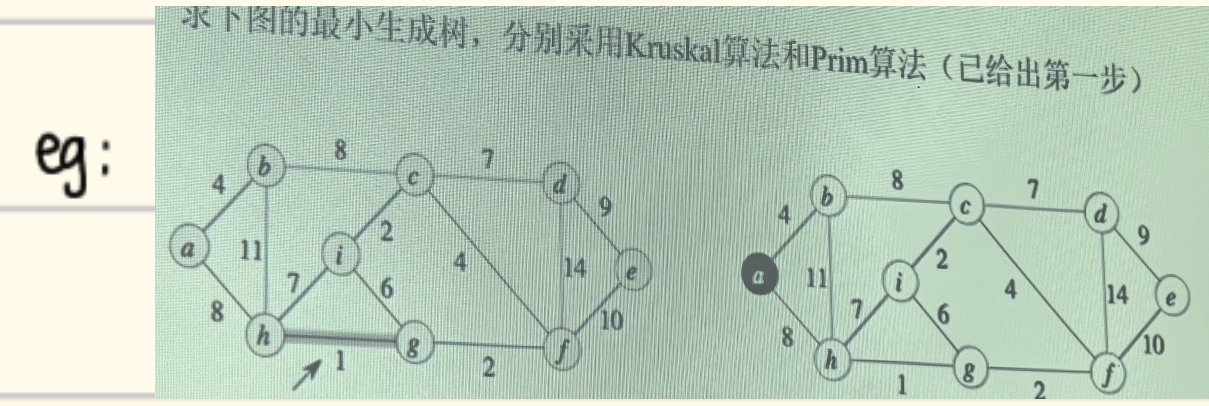
• Prim 算法：

从任一点开始，找出其与外界(非本连通分支的即可)的最小边，然后将这一边的另一个顶点

加入此点集合；接着用这个点集合找与外界的最小边，连续操作即可。

总结： kruskal 算法以边为对象， Prim 算法以点为对象

$O(|E|\log_2|E|)$ ，适用于边少的场景  $O(|V|^2)$ ，适用于点少的场景





• **最短路径**: 图中两点间有很多条路径, 找到使各边权值和最小的路径

• Dijkstra 算法: 求解一个点到图中所有其他顶点的最短路径 (权值非负)

思路: 引入辅助数组  $dist[n]$ , 则  $dist[i]$  为  $v_0$  到终点  $v_i$  的最短路径长度。

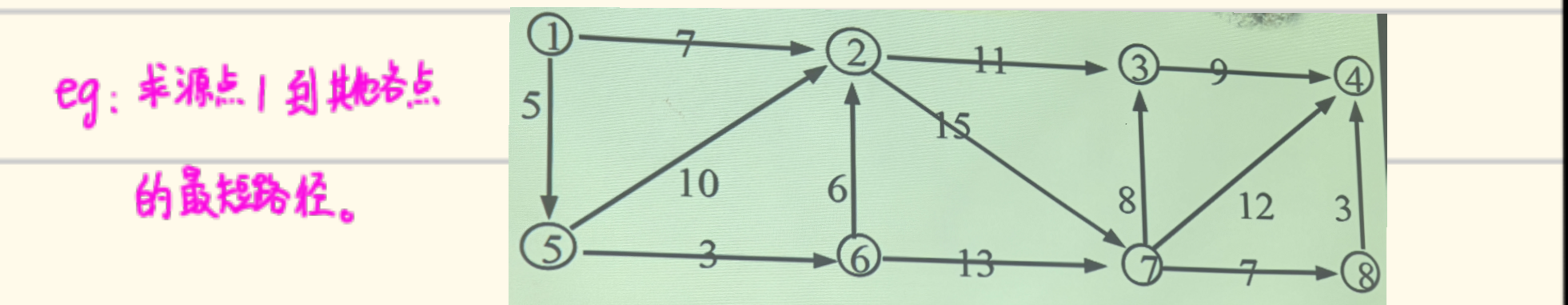
引入顶点集合  $S$ , 保存“当前  $v_0$  可达到的点的集合”,  $S$  初始化为只有  $v_0$ 。

①  $dist$  初始化为  $v_0$  到  $v_i$  的直达边长, 若不直接连通则设为  $\infty$

② 找出最小的  $dist[i]$ , 设此时  $i=k$ , 则  $S=S \cup \{k\}$

③ 更新所有  $dist$ , 即对每一个  $i \in V-S$ ,  $dist[i] = \min\{dist[i], dist[k] + E[k][i]\}$

④ 判断: 若  $S=V$ , 则结束, 否则跳到③



$dist: [2:7, 3:\infty, 4:\infty, 5:5, 6:\infty, 7:\infty, 8:\infty]$        $S: \{1\}$

$dist: [2:7, 3:\infty, 4:\infty, 5:5, 6:8, 7:\infty, 8:\infty]$        $S: \{1, 5\}$

$dist: [2:7, 3:18, 4:\infty, 5:5, 6:8, 7:22, 8:\infty]$        $S: \{1, 5, 2\}$

$dist: [2:7, 3:18, 4:\infty, 5:5, 6:8, 7:21, 8:\infty]$        $S: \{1, 5, 2, 6\}$

$dist: [2:7, 3:18, 4:27, 5:5, 6:8, 7:21, 8:\infty]$        $S: \{1, 5, 2, 6, 3\}$

$dist: [2:7, 3:18, 4:27, 5:5, 6:8, 7:21, 8:28]$        $S: \{1, 5, 2, 6, 3, 7\}$

$dist: [2:7, 3:18, 4:27, 5:5, 6:8, 7:21, 8:28]$        $S: \{1, 5, 2, 6, 3, 7, 4\}$

$dist: [2:7, 3:18, 4:27, 5:5, 6:8, 7:21, 8:28]$        $S: \{1, 5, 2, 6, 3, 7, 4, 8\}$

(结果)

时间复杂度:  $O(n^2)$  (使用最小堆后变为  $O(n \log n)$ )

\* Floyd 算法: 计算每一对顶点之间的最短路径:

思路: 引入辅助数组  $a[n][n]$ ;  $a^{(1)}$  即为  $Edge[i][j]$

$a^{(k)}[i][j]$  表示  $v_i \rightarrow v_j$ , 经过  $v_k$  的最短路径长度。

$a^{(k)}[i][j]$  表示  $v_i \rightarrow v_j$ , 中间顶点序号不大于  $k$  的最短路径长度。

$a^{(n)}[i][j]$  即为任一顶点对  $i \rightarrow j$  的最短路径长度

时间复杂度:  $O(n^3)$

\* **活动网络**

\* DAG 图: 有向无环图 (后续讨论的基础)

• AOV 网络: 用顶点表示活动的网络

顶点表示活动, 有向边  $\langle v_i, v_j \rangle$  表示活动  $i$  必须先于活动  $j$  进行。形成的一定是 DAG 图。

\* 拓扑排序: 用于判断是否为 DAG 图

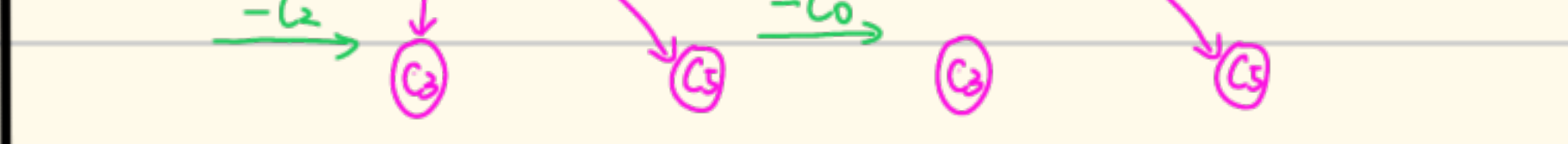
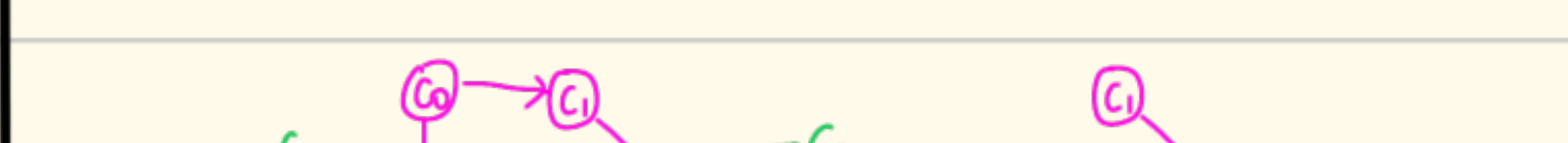
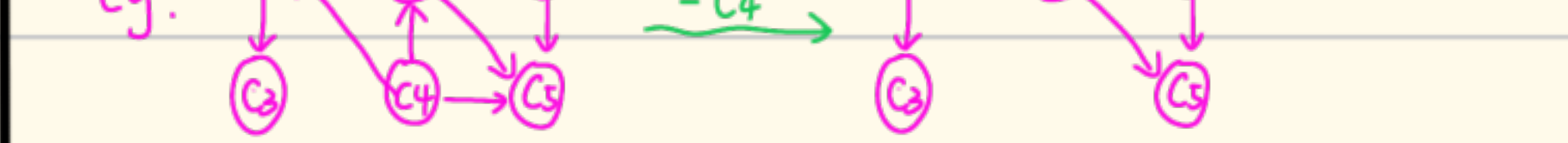
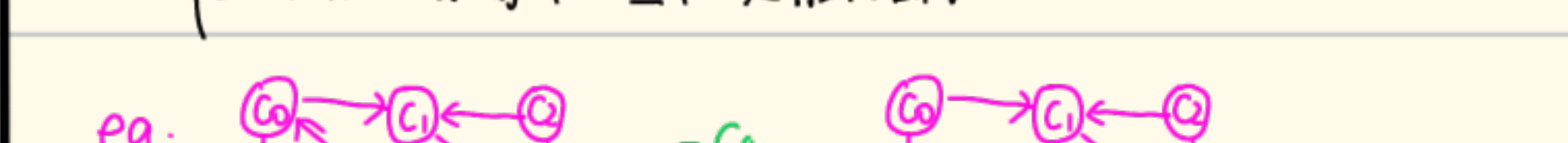
① 在 AOV 网络找一个没有直接前驱的顶点, 并输出

② 从图中删去该点及其发出的所有边

重复①②步, 直至找不到没有直接前驱的顶点。此时, 若:

所有顶点均已输出: 图是 DAG 图

还有顶点没有输出: 图中必定有环结构



故拓扑有序序列为  $C_4, C_2, C_0, C_3, C_1, C_5$ , 该图为 DAG 图

• AOE 网络: 在一个 DAG 图中, 用有向边表示活动, 权值表示活动时间, 顶点表示事件

整个工程只有一个起始点 (入度为 0, 称为源点) 和完成点 (出度为 0, 称为汇点)

只有各条路径上所有活动都完成了, 整个工程才算完成。

完成整个工程所需时间取决于从源点到汇点的最长路径长度 (即: 关键路径)

• 重要的量:

1. 事件  $v_i$  的最早可能开始时间  $Ve[i]$ : 源点  $v_0$  到  $v_i$  的最长路径长度

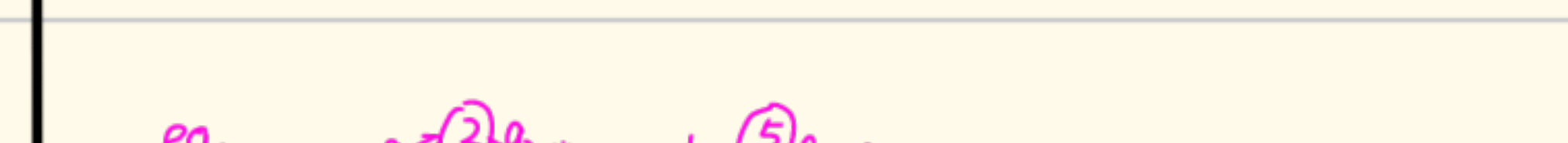
2. 事件  $v_i$  的最迟允许开始时间  $VL[i]$ : 在保证汇点  $v_n$  在  $Ve[n]$  时刻完成

的前提下, 事件  $v_i$  的允许的最迟开始时间

3. 活动  $a_k$  的最早可能开始时间  $Ae[k]$ : 设  $a_k$  在边  $\langle v_i, v_j \rangle$  上, 则  $Ae[k] = Ve[i]$

4. 活动  $a_k$  的最迟允许开始时间  $Al[k]$ : 同理,  $Al[k] = VL[j] - \text{cost}(\langle v_i, v_j \rangle)$

\*\* 关键活动即  $Ae[k] == Al[k]$  的活动



$Ve: [0, 8, 12, 22, 28, 40, 46, 58]$

$VL: [0, 8, 12, 22, 28, 40, 46, 58]$

$Ae: [0, 0, 8, 12, 12, 22, 22, 28, 40, 46]$

$Al: [0, 0, 8, 12, 12, 32, 22, 28, 40, 46]$

故只有  $a_6$  为关键路径